

Docker & n8n — Guided Lab

Containers for OSINT, and an automated pipeline with n8n

Day 3 · infrastructure + automation · from `docker run` to a scheduled OSINT→STIX pipeline

You will learn to stand up disposable infrastructure in seconds, run OSINT tooling inside throwaway containers, then deploy **n8n** and build an automated, agentic OSINT pipeline that feeds a STIX graph. This is the operational backbone under every other Day 3 lab.

Why containers for OSINT? A container is a clean, isolated, **disposable** environment. You spin one up for a case, work in it, and destroy it — nothing persists unless you chose a volume. That volatility is an OPSEC feature (Day 1 SecOps): when you need to be gone, you `docker rm` and the workspace is gone. Same idea as the Day 1 `osint-lab` container, now generalised.

Learning objectives

- Run, inspect and destroy containers; understand images, containers, volumes and ports.
- Stand up a small web server in one command, and see why it is disposable.
- Run an OSINT tool inside a throwaway container and tear it down cleanly.
- Deploy **n8n** with `docker compose`.
- Build and schedule an **agentic OSINT** → **STIX** pipeline in n8n (advanced).

Prerequisites

- A Linux VM (or Docker Desktop on macOS/Windows). Working inside a VM is recommended: it is itself a disposable layer, and it keeps the lab off your host.
- Docker Engine + the Compose plugin: `docker --version` and `docker compose version`.

Part 1 — Docker in 10 commands (25 min)

Run each, and write in your notes what it does.

```
docker run hello-world           # pull + run a test image
docker run -it --rm debian:bookworm-slim bash # throwaway shell; 'exit' destroys it
docker ps                        ;    docker ps -a      # running / all containers
docker images                    # local images
docker pull nginx:alpine         # fetch an image
```

```
docker run -d --name web -p 8088:80 nginx:alpine    # a web server, detached
docker exec -it web sh                             # get a shell inside it
docker logs web                                     # its output
docker stop web ; docker rm web                    # stop + remove
docker system df                                    # what disk is Docker using?
```

1.1 Open `http://localhost:8088` after the `nginx` run. How many seconds from command to a live web server? What does `-p 8088:80` map?

1.2 `--rm` vs no `--rm`: which command left nothing behind, which one you had to `rm`? Relate this to the disposability you want in an investigation.

1.3 Images vs containers vs volumes: define each in one line, using what you just saw.

Part 2 — A disposable web server & workspace (20 min)

The lab ships a `docker-compose.yml`. Bring the demo web server up and down:

```
cd "Day_3/Docker and n8n Lab"
docker compose up -d web                          # serves ./site at http://localhost:8088
# edit ./site/index.html, refresh the page
docker compose down                               # containers gone; the named volume stays
docker compose down -v                            # -v also deletes the volume: nothing
survives
```

2.1 What in this stack survives a `down`? What survives `down -v`? Where would case data go if you wanted it to persist on the host (hint: a bind mount `./work:/work`)?

2.2 Mount a host folder into a container and write a file from inside it:

```
docker run -it --rm -v "$PWD/work:/work" debian:bookworm-slim \
  bash -c 'echo "note from the container" > /work/note.txt'
cat work/note.txt
```

Why is a bind mount the bridge between a disposable container and the evidence you must keep?

Part 3 — Run OSINT tools in throwaway containers (25 min)

Instead of installing tools on your host, run them in a container that vanishes afterwards.

```
# metadata tool, no host install, destroyed on exit
docker run --rm -v "$PWD/work:/work" -w /work \
```

```
ghcr.io/... /bin/sh -c 'exiftool photo.jpg'          # (use the Day 1 osint-lab image)

# or reuse the Day 1 image you built (SecOps Lab)
docker run --rm -it -v "$PWD/work:/workspace" osint-lab:latest \
  zsh -c 'sherlock somehandle --print-found --no-txt'
```

3.1 Run one username or metadata tool from the Day 1 `osint-lab` image against a file in `./work`. Confirm that after the run, `docker ps -a` shows nothing left.

3.2 Why is "tool in a throwaway container" both an OPSEC win (no traces on the host) and a reproducibility win (same image, same result)?

Part 4 — Deploy n8n (20 min)

n8n is a visual workflow automation tool. Deploy it from the shipped compose:

```
docker compose up -d n8n
# open http://localhost:5678 (create the local owner account when prompted)
docker compose logs -f n8n          # watch it boot
```

4.1 Which environment variables in `docker-compose.yml` set the timezone to UTC and disable phone-home telemetry? Why do both matter for an investigator?

4.2 Where does n8n persist its workflows (which volume)? What happens to them on `down` vs `down -v`?

Part 5 — Build the agentic OSINT → STIX pipeline (advanced, 45 min)

Import the shipped workflow and make it real.

```
n8n UI -> Workflows -> Import from File -> workflows/osint_stix_pipeline.json
```

The pipeline: a **weekly trigger** fans out to two **parallel agents** (crt.sh subdomains, Google News RSS); both converge on a **writer** node that maps findings to a **STIX 2.1** bundle; the writer fans out to **load into OpenCTI/TAXII** and to an **alert-on-change** node.

5.1 Replace `<TARGET>` in both HTTP nodes with a domain you are authorised to test. Execute the workflow manually (▶). Inspect the output of each node.

5.2 Read the **Writer** Function node. It mirrors `osint_to_stix.py` from the STIX Lab: one `domain-name` SCO and one `indicator` per subdomain. Add an `ObservedData` object grouping the SCOs.

- 5.3 Read the **Alert on change** node. It stores the previous run's ids in workflow static data and emits only new ones. Which Day 1 principle is this (idempotent monitoring)?
- 5.4 Wire a real notification: add a node (email / webhook / Slack) after the alert node so a new finding pings the team.
- 5.5 Point the **Load** node at a real OpenCTI/TAXII endpoint (or a `httpbin.org/post` stand-in) and confirm the bundle is delivered.

This is the same shape as `backend/agents.py` in the STIX Lab, but visual, scheduled and team-operable. Code when you need control; n8n when you need a pipeline the whole team can see and edit.

Deliverables

One zip `DOCKER_N8N_<team>.zip`:

1. Your notes for Part 1 (the 10 commands) and answers to 1.1–1.3.
2. A screenshot of the demo web server, and of `docker ps -a` showing a clean teardown after Part 3.
3. Your imported n8n workflow (exported back to JSON) with: the added `ObservedData` in the writer, a working notification node, and a delivered bundle (screenshot of a successful run).
4. Answers to 4.1–4.2 and 5.1–5.5.

Grading grid (100 pts)

Item	Pts	What earns the points
Docker fundamentals	20	commands understood; images/containers/volumes defined
Disposability & mounts	15	down vs down -v; bind mount used; OPSEC reasoning
OSINT in throwaway containers	15	a tool run + clean teardown shown
Deploy n8n	15	n8n up; UTC + telemetry env explained; persistence understood
Agentic pipeline	30	parallel agents run; writer emits valid STIX; alert-on-change; notification wired
Deliverable quality	5	reproducible, screenshots, exported workflow

Optional Extensions (Bonus)

- **Everything in the VM:** run n8n, the web server and the Day 1 `osint-lab` on one VM; snapshot it clean, and prove you can restore to the snapshot after a case (Day 1 discipline).
- **Compose the whole stack:** add the Day 1 `osint-lab` image and a small database to the compose file so one `up` gives you a full investigation environment.
- **STIX validation in-line:** add a node that POSTs the bundle to a small validation service before the load node.
- **Secrets:** move the OpenCTI token to n8n credentials (never in the node), and the API keys to environment variables — the Day 1 code-OPSEC rule.